God of Prompt
@godofprompt

karpathy's second brain: how to build it

33 234 1.6K 435K

100K people bookmarked @karpathy's post:

> **Andrej Karpathy** @karpathy · Apr 2
> LLM Knowledge Bases
>
> Something I'm finding very useful recently: using LLMs to build personal knowledge bases for various topics of research interest. In this way, a large fraction of my recent token throughput is going less into manipulating code, and more into manipulating
> Show more
>
> 2.6K 8.1K 53K 18M

Then he dropped a full GitHub Gist. 5,000+ stars. 1,400+ forks. Two days.

Most people will bookmark that too. Then do nothing.

Not because it's hard. Because nobody gave them the exact prompts.

I'm going to fix that.

I'll walk you through the full system, hand you copy-paste prompts for every step, and tell you where this breaks so you don't waste a weekend on something that falls apart at scale.

"The human's job is to curate sources, direct the analysis, ask good questions, and think about what it all means. The LLM's job is everything else." — Andrej Karpathy

---

The Concept (60-Second Version)

You have knowledge scattered everywhere. Articles saved in 4 apps. Bookmarks from 2023 you'll never revisit. Notes from meetings that live in a folder you forgot existed.

Right now, when you ask AI a question about your stuff, it starts from zero every time. Upload docs, ask a question, get an answer. Next session? It's forgotten everything. That's how ChatGPT file uploads, NotebookLM, and most RAG systems work. Zero accumulation.

Karpathy's idea flips this.

Instead of the AI searching your raw files every time, the AI reads your sources once and compiles a structured wiki. Summaries, cross-references, connections between ideas, contradictions flagged.

All maintained by the AI. All in simple markdown files.

Next time you ask a question, the AI doesn't dig through raw documents. It reads the wiki it already built.

The connections are already there.

The synthesis already reflects everything you've read.

Every new source you add makes the wiki richer. Every question you ask can get filed back in. Knowledge compounds instead of resetting.

His result: ~100 articles, ~400,000 words on a single research topic. He didn't write a word of it. The AI wrote, linked, categorized, and maintained all of it.

No database. No embeddings. No vector store. Just folders and text files.

Why Should You Care?

Three use cases that matter right now:

If you're a creator or marketer, this is a content research engine. Dump competitor breakdowns, trending articles, audience insights into raw/. The wiki surfaces patterns and angles you'd never find manually.

If you're a founder or consultant, this is your second brain for client work, market research, or competitive analysis. Every report you generate feeds back into the system. By month 3, your AI knows your domain better than most hires.

If you're a student or researcher, this is what Karpathy actually built it for. Deep research across dozens of papers, with the AI tracking how ideas connect, where authors disagree, and what gaps remain.

*This can also be used for a lot of business R&D workflows.

Before We Build: What You Need

→ Any AI coding tool that reads local files (Claude Code, Cursor, Codex, or similar)

→ A text editor (Obsidian recommended, but VS Code, Notepad, anything works)

→ 10+ source documents on a topic you care about

→ 30 minutes for initial setup, then 10 minutes per source after that

That's it. No special software. No accounts to create. No plugins to install.

The rest of this article is the build. 7 steps. Every step has the exact prompt you'll paste into your AI. Follow them in order.

Step 1: Create the Folder Structure (2 Minutes)

Create this anywhere on your machine:

```
my-knowledge-base/
├── raw/           # Your source material. AI reads but never modifies.
│   └── assets/    # Images, screenshots, diagrams
```

```
├── wiki/          # AI-maintained wiki. You read. AI writes.
├── outputs/       # Reports, analyses, answers from queries
└── CLAUDE.md      # The schema file that makes this whole thing work
```

Three folders, one file. If you're spending more than 2 minutes here, you're overthinking it.

Step 2: Write Your Schema File (The Step Everyone Skips. Don't.)

The schema is the difference between a generic chatbot and a disciplined wiki maintainer.

It tells your AI what the knowledge base is about, how to organize it, and what to do when you add sources, ask questions, or run maintenance.

Every other guide gives you a 10-line template. Here's the full production schema, based on Karpathy's gist, engineered for real use:

```
# Knowledge Base Schema

## Identity
This is a personal knowledge base about [YOUR TOPIC].
Maintained by an LLM agent. The human curates sources and asks questions. The

## Architecture
- raw/ contains immutable source documents. NEVER modify files in raw/.
- wiki/ contains the compiled wiki. The LLM owns this directory entirely.
- outputs/ contains generated reports, analyses, and query answers.

## Wiki Conventions
- Every topic gets its own .md file in wiki/
- Every wiki file starts with YAML frontmatter:
  ---
  title: [Topic Name]
  created: [Date]
  last_updated: [Date]
  source_count: [Number of raw sources that informed this page]
  status: [draft | reviewed | needs_update]
  ---
- After frontmatter, a one-paragraph summary
- Use [[topic-name]] for internal links between wiki pages
- Every factual claim cites its source: [Source: filename.md]
- When new info contradicts existing content, flag explicitly:
  > CONTRADICTION: [old claim] vs [new claim] from [source]

## Index and Log
- wiki/index.md lists every page with a one-line description, by category
- wiki/log.md is append-only chronological record
- Log entry format: ## [YYYY-MM-DD] action | Description
  (Actions: ingest, query, lint, update)

## Ingest Workflow
When processing a new source:
1. Read the full source document
2. Discuss key takeaways with user
```

```
3. Create or update a summary page in wiki/
4. Update wiki/index.md
5. Update ALL relevant entity and concept pages across the wiki
6. Add backlinks from existing pages to new content
7. Flag any contradictions with existing wiki content
8. Append entry to wiki/log.md
9. A single source should touch 10-15 wiki pages

## Query Workflow
When answering a question:
1. Read wiki/index.md first to find relevant pages
2. Read all relevant wiki pages
3. Synthesize answer with [Source: page-name] citations
4. If answer reveals new insights, offer to file it back into wiki/
5. Save valuable answers to outputs/

## Lint Workflow (Monthly)
Check for:
- Contradictions between pages
- Stale claims superseded by newer sources
- Orphan pages with no inbound links
- Concepts mentioned but never explained
- Missing cross-references
- Claims without source attribution
Output: wiki/lint-report-[date].md with severity levels

## Focus Areas
[List 3-5 topics this knowledge base covers]
```

Copy this. Customize the focus areas. Drop it in your project root as CLAUDE.md.

Step 3: Fill Your Raw Folder (10 Minutes of Dumping, Zero Organizing)

Open raw/ and dump everything in:

→ Copy-paste articles into .md or .txt files

→ Export notes from whatever app you're using now

→ Save screenshots and diagrams to raw/assets/

→ Paste in research papers, PDFs, competitor breakdowns

→ Dump bookmarks you've been hoarding for months

Don't organize it. Don't rename anything. Don't clean it up. That's the AI's job.

Pro tip from Karpathy: The Obsidian Web Clipper browser extension converts any web article to markdown in one click.

Set a hotkey (Settings → Hotkeys → "Download attachments") to pull all images locally so the AI can reference them.

If you don't use Obsidian, copy-paste from your browser works fine.

The goal is volume. Not perfection.

Step 4: Run Your First Ingest

Open your AI agent. Point it at your project folder. Paste this:

INGEST PROMPT:

```
"Read the schema in CLAUDE.md. Then process [FILENAME] from raw/. Read it full
```

Start with one source at a time. Karpathy does the same. Read the summaries. Check the updates. Guide the AI on what to emphasize. This produces dramatically better results than batch-processing everything at once.

After 5-10 sources, your wiki/ folder will have an index, a log, and 15-30 interconnected pages.

That's when things click.

Step 5: Start Querying Your Knowledge Base

Once you have 10+ wiki pages, the system becomes genuinely useful. Paste this:

QUERY PROMPT:

```
"Read wiki/index.md. Based on what's in the knowledge base, answer: [YOUR QUES
```

Questions that extract the most value:

→ "What are the three biggest gaps in this knowledge base?"

→ "Which sources disagree with each other, and on what?"

→ "What should I research next based on what's here?"

→ "Write a 500-word briefing on [topic] using only wiki content"

→ "What connections exist between [concept A] and [concept B]?"

The critical loop: good answers should be filed back into the wiki.

A comparison, an analysis, a connection you discovered.

These compound in the knowledge base just like ingested sources do.

Every question makes the next answer better.

Step 6: Run Monthly Health Checks

This is the step nobody does. It's the step that prevents the whole system from slowly rotting. Paste this:

LINT PROMPT:

```
"Run a full health check on wiki/ per the lint workflow in CLAUDE.md. Output t
```

Why this matters: when the AI writes something slightly wrong and you save it back, the next answer builds on the wrong thing.

Two months later, you have five pages reinforcing the same error. Health checks catch this before it snowballs.

One check per month. Ten minutes of your time. Non-negotiable if you want the system to stay trustworthy.

Step 7: Let It Compound

This is where the system earns its keep.

After 4-6 weeks of consistent use, you're not just searching notes.

You're querying a structured knowledge system that understands connections between your sources better than you do.

Three ways to accelerate the compounding:

File exploration outputs back: When the AI generates a comparison or analysis you find valuable, save it into wiki/ or outputs/.

Karpathy says his own explorations and queries "always add up" in the knowledge base.

Add visual outputs: Have the AI render answers as markdown tables, charts, or slide decks (Marp format).

These become reusable assets, not throwaway chat messages.

Version control everything: Your wiki is just markdown files.

Initialize a git repo. You get full history, branching, and the ability to undo anything the AI messes up.

Okay. That's the build. Now here's the part nobody else will tell you.

Where This System Breaks (The Honest Version)

This is a nascent pattern, not a finished product. Karpathy himself called it "a hacky collection of scripts" and said there's room for a real product.

Here's what you need to know before trusting your knowledge to it:

Context Window Ceiling.

Karpathy's wiki works at ~100 articles and ~400K words. But even 128K-token context windows only hold ~96K words. The AI reads selectively through the index, which means it can miss things. Research shows LLMs suffer from "lost in the middle" effects where information in the center of long inputs gets deprioritized. Your query results will have blind spots. Accept this.

Error Compounding.

The AI writes a wiki page with a subtle mistake. You query against it. The mistake enters your answer. You file that answer back. Now two pages reinforce the same error. Monthly linting helps, but the AI doing the linting has the same blind spots as the AI that made the error. This is the

single biggest risk. One commenter on Karpathy's gist nailed it: "When outputs get filed back, errors compound too."

Hallucination Doesn't Disappear.

The wiki approach reduces hallucination because the AI grounds answers in your sources. But it doesn't eliminate it. The AI can still synthesize connections that don't exist in the source material. And because the wiki looks authoritative (clean markdown, cross-references, citations), you're more likely to trust incorrect information. Don't.

Cost Isn't Zero.

Every ingest, every query, every lint check costs tokens. A single source that touches 10-15 pages can run $2-5 in API calls with frontier models. 50 sources is $100-250 just for ingestion. Cheaper than a research assistant. Not free.

It Doesn't Scale to Enterprise.

Karpathy says the index-file approach works without RAG at ~100 articles. At 10,000+ sources, this pattern breaks. The index grows too large. Consistency across thousands of pages becomes impossible. You'll need the infrastructure this system was designed to avoid. Know the ceiling.

Single-Model Blind Spots.

Your entire wiki is one model's interpretation of your sources. That model has biases and tendencies. For high-stakes decisions, one gist commenter suggested running queries through 4+ models independently, then comparing agreement. More robust. Also 4x the cost.

What To Do About It

→ Error compounding: Monthly lint checks. Cross-check critical claims manually. Never trust the wiki blindly on high-stakes decisions.

→ Context limits: Keep each wiki focused on one domain. Multiple domains? Multiple knowledge bases.

→ Cost: Use frontier models for ingest and complex queries. Cheaper models for simple updates.

→ Hallucination: The schema above requires source citations on every claim. If a page makes a claim without [Source: filename], linting will flag it.

→ Scale: Accept this is a personal tool, not enterprise infrastructure. If you outgrow it, that's a good problem.

Why It Still Matters

Despite all the above, this is the most practical personal knowledge system available right now.

The reason is dead simple: humans abandon wikis because maintenance grows faster than value.

You start organizing, it feels great for two weeks, then the upkeep kills motivation and you never touch it again.

LLMs don't get bored. They don't forget to update a cross-reference. They can touch 15 files in one pass without complaining.

Lex Fridman confirmed he runs a similar setup.

He generates interactive HTML visualizations and creates "mini-knowledge-bases" he loads into voice mode for 7-10 mile runs.

Elvis Saravia from DAIR.AI has been building LLM knowledge bases for AI research curation.

Multiple open-source implementations hit GitHub within 48 hours of Karpathy's gist.

This isn't an experiment anymore.

It's becoming standard practice for anyone doing serious research.

Your Complete Prompt Library (Copy Everything)

Every prompt from this article, collected in one place:

SCHEMA: Copy the full CLAUDE.md template from Step 2.

INGEST (one source):

```
"Read the schema in CLAUDE.md. Process [FILENAME] from raw/. Read it fully, di
```

INGEST (batch, less supervised):

```
"Read CLAUDE.md. Process all unprocessed files in raw/ sequentially. For each:
```

QUERY:

```
"Read wiki/index.md. Answer: [QUESTION]. Cite wiki pages. If this answer is wo
```

LINT:

```
"Run a full health check on wiki/ per the lint workflow in CLAUDE.md. Output t
```

EXPLORE:

```
"Read wiki/index.md and identify the 5 most interesting unexplored connections
```

BRIEF:

```
"Based on everything in wiki/, write a 500-word executive briefing on [TOPIC].
```

Go Build It

The difference between bookmarking Karpathy's gist and benefiting from it is one afternoon.

Pick your topic. Create the folders. Copy the schema.

Drop in what you already have. Run your first ingest.

Then do it again tomorrow with another source.

And next week with five more.

The wiki gets smarter every time. That's the whole point.

Three folders. One schema.

An AI that does the grunt work you'd never do yourself.

Stop collecting bookmarks. Start compiling knowledge.